

JARVIS AI Voice Assistant

Comprehensive Technical Documentation & Code Architecture Breakdown

1. OVERVIEW

JARVIS is an automated Python-based voice assistant designed to interact with users using natural speech recognition and high-quality Text-to-Speech (TTS). It integrates **Google Gemini AI (LLM)**, **Xweather REST API**, and **Wikipedia** to perform real-time information retrieval, multi-turn reasoning, and execution of voice command actions.

2. CORE FEATURES & CAPABILITIES

- Voice I/O** Hands-free control using PyAudio / SpeechRecognition and Windows Native SAPI.SpVoice.
- Weather Engine** Dual-query location extraction matching real-time meteorological metrics via Xweather.
- LLM Integration** Connected directly to `gemini-3.6-flash` via the official Google GenAI SDK.
- Knowledge Lookup** Instant 2-sentence query extraction directly from Wikipedia pages.
- TTS Sanitization** Automatic Regex string cleaning (strips URLs, Markdown syntax, and non-alphanumeric chars) before audio speech rendering.

3. ARCHITECTURE & CODE STRUCTURE ANALYSIS

3.1 Initialization & Environment Configuration

The script starts by resolving local paths and invoking `dotenv.load_dotenv()` to safely import sensitive API keys from the `.env` file. The client object `genai.Client` is instantiated with the `GEMINI_API_KEY`. Native Windows speech engine dispatching occurs via `win32com.client.Dispatch("SAPI.SpVoice")`.

3.2 Core Functionality & Module Breakdown

FUNCTION NAME	PRIMARY RESPONSIBILITY	KEY LOGIC / DEPENDENCIES
<code>speak(text)</code>	Converts raw strings into clean spoken voice output.	Uses Regex regex patterns to strip markdown (<code>*</code> , <code>_</code> , <code>#</code>) and web links before sending text to SAPI voice synthesizer.
<code>listen()</code>	Captures live audio input from the user's default microphone.	Employs <code>sr.Recognizer</code> with ambient noise suppression (1 sec) and Google Web Speech API speech-to-text decoding.
<code>get_weather(location)</code>	Queries Xweather REST API for live conditions.	Executes target city lookup with automatic fallbacks (e.g., appending country code <code>,India</code> or <code>,IN</code> if first match fails).
<code>extract_location(command)</code>	Parses city/place entities from raw command text.	Runs regex matching against trigger keywords like <i>"weather in"</i> or <i>"temperature of"</i> .
<code>jarvis()</code>	Main runtime execution loop.	

FUNCTION NAME	PRIMARY RESPONSIBILITY	KEY LOGIC / DEPENDENCIES
		Evaluates incoming commands against routing conditions (Exit, Weather, Wikipedia, or LLM General Query fallback).

4. WORKFLOW DIAGRAM & ROUTING LOGIC



5. FUNCTION-BY-FUNCTION TECHNICAL CODE BREAKDOWN

5.1 Text-to-Speech & Cleaning (`speak`)

Raw strings from LLMs or web APIs often contain symbols, URLs, or markdown elements like asterisks (`*`) and code blocks. The `speak()` function ensures the audio engine does not attempt to read out punctuation literally:

```
text = re.sub(r"[*_`#>-]", "", str(text))      # Strip Markdown formatting
text = re.sub(r"https?://[S+|www\.\S+]", "", text)  # Strip URLs
text = re.sub(r"^[a-zA-Z0-9\s.,!?\-]", "", text)    # Keep standard punctuation & letters
speaker.Speak(text)                               # SAPI Voice Output
```

5.2 Weather Retrieval Logic (`get_weather`)

The function constructs dynamic parameters for the Xweather API, featuring robust multi-try query fallbacks for unambiguous location resolution.

Automatic Fallback Strategy

When a single city name is provided without a country specifier (e.g., `"Ahmedabad"`), the engine automatically tries sequential queries: `["Ahmedabad", "Ahmedabad,India", "Ahmedabad,IN"]` to maximize request success.

5.3 Google Gemini AI Integration (`jarvis` Main Loop)

Unrecognized requests flow into the standard LLM handler using the Google GenAI Python SDK. The assistant passes full text prompt contents directly to `gemini-3.6-flash` :

```
response = client.models.generate_content(  
    model="gemini-3.6-flash",  
    contents=command  
)  
speak(response.text)
```

6. ENVIRONMENT SETUP & EXECUTION GUIDELINES

6.1 Prerequisites & Installation

```
pip install requests SpeechRecognition wikipedia pywin32 python-dotenv google-genai
```

6.2 Required Environment Variables (`.env`)

```
GEMINI_API_KEY=your_gemini_api_key_here  
XWEATHER_CLIENT_ID=your_xweather_client_id  
XWEATHER_CLIENT_SECRET=your_xweather_client_secret
```